

腾讯 AI Coding 大模型编程（2607期）实习证明考核报告

课程：【2607期】腾讯 AI Coding 大模型编程

学员：何宇翔

项目：Lumio Music — HarmonyOS 本地离线音乐播放器

实训周期：2026年7月28日 ~ 2026年8月14日

报告日期：2026-08-14

1.1 自我介绍（约150字）

我是2025级的何宇翔，目前就读于桂林电子科技大学的网络工程专业。在参加本次实训前，我已具备一定的编程基础（接触过C++、C、Cangjie），但对 HarmonyOS / ArkTS 生态几乎完全陌生。报名腾讯 AI Coding 实训，一方面是希望把课堂所学的软件工程方法落地到一个真实可运行的产品中，另一方面也想亲身体验大模型辅助编程的完整 workflow，为将来进入行业开发做准备。实训前，我给自己的定位是「能写代码，但缺乏产品视角和工程闭环经验」的开发者；经过三周高密度实战，这个自我认知已经被彻底刷新。

1.2 各阶段学习总结（约800-1200字）

本次实训共分为三个阶段，我把学习经历和项目推进的里程碑对应起来，而不是按天流水账记录。

1.2.1 阶段一：AI 辅助编程入门与开发环境搭建（第一周 7/28 ~ 7/31）

主要接触的内容与任务

第一周聚焦「工具链」与「协作方式」。我完成了 DevEco Studio、CodeBuddy、CloudStudio 的环境配置，生成了第一个 Hello World 工程；同时学习了 Skill 与 MCP 的概念，配置了 `harmonyos-developer`、`harmonyos-reviewer`、`harmonyos-pm` 等专家 Skill，接入了 GitHub MCP，让 AI 能够理解 HarmonyOS 项目的上下文。

具体学到了什么

- Skill 是「场景化指令封装」，可以把高频任务（如代码审查、PRD 规划）固化成可复用的 AI 能力；
- MCP（Model Context Protocol）是 AI 与外部系统（文件、数据库、GitHub 等）交互的标准接口；
- HarmonyOS 构建链非常依赖环境一致性：Node 版本、Java 版本、`hvm` 守护进程、签名证书缺一不可。

在项目中的运用

我直接把这些能力用到了 Lumio Music 的工程搭建中：用 `harmonyos-reviewer` 做静态合规扫描，用 GitHub MCP 管理远程仓库，用 CloudStudio 做云端验证。环境配置不再是「一次性动作」，而是被写进了可复用的构建脚本。

难点与解决

最大的卡点是 `hvm` 构建随机崩溃。最初我以为是代码问题，排查了一整天后发现根因有两个：一是系统 PATH 中残留的 Oracle Java 损坏；二是 WorkBuddy 的 `genie-safe-delete.cjs` 通过 `NODE_OPTIONS` 注入，拦截了 `hvm` 清理构建产物。我通过隔离 PATH 到 DevEco 自带的 JBR OpenJDK、并在构建前置空 `NODE_OPTIONS`，最终写出了一份稳定的 `build_hap.sh`，后续所有 HAP 包都通过它一键签名出包。

1.2.2 阶段二：产品需求分析与 AI 拆解实战（第二周 8/3 ~ 8/7）

主要接触的内容与任务

第二周进入「产品思维」训练：学习用户故事、功能优先级、边界定义，并用 AI 辅助撰写 PRD、拆解功能模块；同时通过第 4 课的案例分享，了解了 AI Coding 在不同场景下的协作模式与常见坑点。

具体学到了什么

- 需求不能只描述「我想要什么」，而要写清楚「谁在什么场景下，完成什么目标，验收标准是什么」；
- 用 AI 写 PRD 时，必须给它足够的上下文（目标平台、API 版本、约束条件），否则它会「hallucinate」出不存在的 API；
- 从需求到架构的完整链路：需求 → 功能模块 → 数据模型 → 接口/页面 → 测试用例。

在项目中的运用

我选定 Lumio Music 作为第三周实战产品：一款纯本地、无需账号、无需网络的 HarmonyOS 音乐播放器。在 AI 辅助下完成了 `docs/PRD_Lumio_Music.md`（34 个功能需求 FR、7 个非功能需求 NFR、风险矩阵）和 `docs/功能模块拆解表.md`（24 个一级模块）。同时输出了两轮实施计划，把大目标拆成了可执行的任务队列。

难点与解决

最初我贪心地想在一周内做「在线音乐 + 本地播放 + 社交分享」，结果 PRD 越写越膨胀。导师和 AI 评审都提示我要守住边界。我用 MoSCoW 方法重新排序，把「本地导入、播放、收藏、歌单」定为 Must have，把在线功能和社交功能果断放到 Won't have，项目才变得可控。

1.2.3 阶段三：AI Coding 产品实战与上线部署（第三周 8/10 ~ 8/14）

主要接触的内容与任务

第三周是完整的产品实战周：用提示词驱动应用骨架生成，实现核心功能，进行 UI 美化与 Bug 修复，补充单元测试，最终生成可访问链接，同时完成项目 README 与 AI 协作复盘报告。

具体学到了什么

- 前端页面、后端接口、本地数据存储在 ArkTS 中如何分工；
- C++ NAPI 与 ArkTS 的交互边界：哪些逻辑适合下沉到原生层（音频元数据解析），哪些必须留在 ArkTS（UI 状态管理）；
- HarmonyOS 的权限最小化原则、AVSession 媒体会话、桌面卡片 FormAbility、后台播放保活机制；
- 单元测试与代码审查不是「锦上添花」，而是保证原型能稳定运行的底线。

在项目中的运用

我在这一周内落地了 Lumio Music 的核心闭环：

- 播放控制：`AudioRendererController` + `AVSessionController`；
- 数据持久化：`MusicStore`（`dataPreferences`）+ `SettingsStore`；
- 原生解析：`cpp/audio_metadata.cpp` 真实解析 FLAC / MP3 / MP4 元数据；
- 体验细节：歌词 Canvas 渲染、一镜到底共享元素动画、长按选项栏、拖拽排序歌单。

最终交付了签名 HAP、演示视频 `Video/Demo.mp4`、以及 45 条单元测试（核心逻辑语句覆盖率 >70%）。

难点与解决

这一阶段的卡点集中在两个地方：

1. **C++ 元数据解析器的边界问题**：最初用 `uint32_t` 表示 atom 长度，遇到大文件时发生 32 位回绕，导致堆越界。我通过审查报告定位到 6 个 P0 缺陷，全部改为 `size_t` / `uint64_t` 并补全边界校验，最终通过真实音频验证。
2. **本地 Git 对象库损坏**：提交前发现本地仓库缺失约 108 个对象（含 commit `f9c2748`），差点影响最终提交。我通过确认 GitHub origin 健康、重新 `git fetch origin` 并修复本地引用，把代码完整同步回本地。

1.3 学习方法分享（约300-500字）

获取新知识的渠道

我最常用的渠道是「官方文档 + CodeBuddy 专家代理 + 课程回放」三件套。遇到 HarmonyOS API 不会用时，先查官方 ArkTS API 文档建立基础认知；再把具体场景抛给 `harmonyos-developer` 生成代码骨架；遇到编译器报错或权限问题时，切换到 `harmonyos-reviewer` 做合规扫描。课程回放则用来补全系统性的方法论（如 PRD 结构、部署流程）。

排查问题的路径

我的排查路径基本是：

1. 复现问题并记录最小触发条件；
2. 把报错日志粘贴给 AI，让它解释错误码和可能根因；
3. 在隔离环境（最小 Demo 或单元测试）中验证假设；
4. 修复后回写到文档或脚本，避免二次踩坑。

比如 `build_hap.sh` 的崩溃，我通过逐步注释脚本、对比成功与失败的环境变量，最终锁定 `NODE_OPTIONS` 和 Java PATH。

笔记与知识沉淀

我坚持用 Markdown 做笔记：

- 每日工作记录写入 `.workbuddy/memory/YYYY-MM-DD.md`；
- 长期项目规范写入 `.workbuddy/memory/MEMORY.md`；
- 设计决策、PRD、审查报告、复盘报告统一放在 `docs/` 目录。

这种结构让我在三周后仍能快速回溯任何技术决策的来龙去脉。

团队协作与同步

由于我是独立开发者，「团队」更多体现为多 Agent 协作：让 `pm` 做规划、`developer` 做实现、`reviewer` 做审查。同时我把代码镜像到 Gitea 私有仓库，作为本地 Git 损坏时的兜底备份。代码提交遵循小步快跑原则，每天至少有 3~5 个 commit，确保随时可回滚。

放弃过的方法

我最初尝试手动维护所有国际化字符串，但 HarmonyOS 的 `i18n.t` 不支持占位符，三语兼容非常痛苦。后来改用 AI 批量生成 + 人工复核，效率提升很多。这让我意识到：不是所有细节都值得手写，关键是用好 AI 并保留最终校验环节。

1.4 实习收获与感受（约500-800字）

专业 / 知识层面

实训前，我对「真实项目」的想象是：先写功能代码，再简单测一下就能上线。实训后才发现，真正的工作量有 80% 花在边界条件、错误处理、兼容性、构建脚本和文档上。Lumio Music 看起来只是一个本地音乐播放器，但要让它在 HarmonyOS 上稳定运行，我必须同时理解 ArkTS 的状态管理、C++ 的内存安全、MediaKit 的音频渲染、AVSession 的媒体会话、以及 hvigor 的构建机制。

我对 HarmonyOS 生态的认知也发生了转变：它不是一个「换皮的 Android」，而是一套从语言（ArkTS）、UI 框架（ArkUI）、编译器到分布式能力都重新设计的系统。那些看似「反直觉」的限制（比如 `@Component` 上普通 `get` 访问器会被丢弃、禁止解构声明、禁止 `any` 类型），本质上都是为了保障端侧性能和稳定性。理解这些约束后，我写代码时会更主动地规避红线，而不是等编译器报错再修。

职业素养层面

这次实训让我对「时间管理」和「责任意识」有了切身体会。课程采用「持续循环 + 截止时间」的节奏，每天北京时间 06:30 是一个迭代节点。有好几次我为了赶在节点前闭环一个功能，熬夜到很晚。印象最深的是修复 `build_hap.sh` 那晚：我反复测试了十几种 PATH 和 `NODE_OPTIONS` 组合，最终把稳定脚本提交到仓库。这件事让我意识到，工程能力不仅体现在写出功能，更体现在为他人留下可复现、可维护的环境。

自我认知层面

通过实训，我发现自己的优势是**快速学习新技术并把碎片信息整合成可运行系统**。面对一个全新的 HarmonyOS 项目，我能在短时间内搭出骨架、定位问题、写出文档。但短板也很明显：**对测试和边界条件的重视程度不够，容易过度乐观估计开发时间**，有时会因为相信 AI 输出而忽略人工复核。

未来半年，我计划做一件具体的事来补短板：**每周阅读一篇 HarmonyOS 官方 ArkTS / ArkUI 文档，并用当周所学独立实现一个小组件**（如自定义动画、列表性能优化、Form 卡片）。通过高频小练习，把「能跑」变成「写得对、写得稳」。

1.5 项目实习反馈与建议（约200-400字）

内容难度与业务场景

整体难度适中，尤其适合有一定编程基础、但缺少完整产品经验的同学。HarmonyOS 作为实训平台很有价值，因为它强制要求严格的类型安全和工程规范；但网络上可参考的第三方案例和开源项目相对较少，遇到冷门 API 问题时需要花更多时间翻官方文档。

指导方式与课程安排

三周的节奏紧凑但合理：第一周打工具基础，第二周练产品思维，第三周完整实战。如果能在第三周产品实战的中间增加一次集中答疑或 checkpoint，会帮助更多同学及时调整方向，避免最后两天赶工。

给下一期同学的建议

不要等到最后才整理文档和复盘。建议从第一天开始就：

1. 每天写一条工作日志；
2. 每个重要决策都留一份 Markdown 记录；
3. 小步提交代码，避免本地仓库损坏后无法恢复；
4. 让 AI 写代码，但自己必须能解释每一行为什么这样写。

总体评价

这是一次「把课堂知识压进真实项目」的高密度训练。虽然过程中踩了很多坑，但每解决一个问题，我对工程化开发的理解就深一层。非常感谢课程团队提供的学习机会。

1.6 附录

附录 A：主要交付物清单

课程要求	交付物路径	说明
第3课：PRD 文档	docs/PRD_Lumio_Music.md	34 个 FR / 7 个 NFR / 风险矩阵
第3课：功能模块拆解表	docs/功能模块拆解表.md	24 个一级模块
第4课：案例分析报告	docs/代码审查报告_PRD落地.md + 代码审查报告_第二轮增强.md	含 6 个 P0 缺陷修复记录
第4课：开发计划	docs/实施计划_PRD落地.md + 实施计划_第二轮增强.md	两轮迭代计划
第5课：可运行产品原型	entry/build/default/outputs/default/entry-default-signed.hap	签名 HAP，可安装运行
第6课：AI 协作复盘	docs/AI协作复盘报告.md	3600 字复盘
第6课：项目 README	README.md	功能、架构、构建说明
第6课：产品演示	Video/Demo.mp4	演示视频

附录 B：关键数据

- 代码审查轮次：5 轮
- 静态审查结果：0 ERROR / 0 WARNING
- 功能需求实现：34 / 34（100%）
- C++ UB 缺陷修复：6 项
- 单元测试：45 条，核心逻辑语句覆盖率 >70%
- 项目开源地址：github.com/a703201/Lumio_Music